ЛЕКЦІЙНИЙ КУРС

Основи ASP.NET Core

Платформа · конвеєр обробки запиту · REST-архітектура

.NET 10 · 21 практичний приклад в одному solution · **AspNet-Basics**

Про курс

Мета — не завчити рецепти, а зрозуміти, що відбувається з HTTP-запитом від моменту, коли його прийняв сервер, до моменту, коли клієнт отримав відповідь. Курс складається з кількох шарів:

1. **Платформа** з чого складається веб-застосунок на .NET — хост, HTTP-сервер Kestrel, конвеєр middleware — і як він стартує
2. **Конвеєр обробки запиту** middleware — головна ідея ASP.NET Core: порядок виконання, розгалуження, куди вписувати наскрізні задачі
3. **Побудова API** endpoint-и на класах-контролерах: маршрутизація атрибутами, прив'язка параметрів, валідація, фільтри — поверх того самого конвеєра
4. **REST-архітектура** як проєктувати ресурси й URI, коди відповідей, представлення, версії, гіпермедіа
5. **Якість** документація OpenAPI, інтеграційні тести через WebApplicationFactory, базова безпека з JWT

- *21 приклад в одному solution: один проект = один урок, з коментарями, файлом NOTES.md і готовими HTTP-запитами.*

Структура курсу — 9 модулів

Модуль	Тема	Приклади
1	Хостинг і платформа	01–02
2	Конвеєр обробки запиту (middleware)	03–06
3	Dependency Injection і конфігурація	07–08
4	Маршрутизація	09
5	Web API на контролерах	10–11
6	MVC глибше: [ApiController], фільтри	12–13
7	REST по-справжньому	14–18
8	Документація і тести	19–20
9	Безпека (базово)	21

Як влаштований кожен приклад

Program.cs поділено на три фази життєвого циклу застосунку

```
var builder = WebApplication.CreateBuilder(args);

// ---- 1 · СЕРВІСИ (реєстрація в DI) ----
builder.Services.AddCatalog();

var app = builder.Build();

// ---- 2 · КОНВЕЄР (middleware) ----
app.UseAuthentication();

// ---- 3 · ЗАПИТИ (endpoints / обробники) ----
app.MapGet("/products", (ICatalog c) => c.GetProducts());

app.Run();
```

- Program.cs — код прикладу з навчальними коментарями
- NOTES.md — теорія, чеклист, посилання на документацію
- *.http — готові запити для VS / Rider / VS Code
- Кожен приклад слухає власний порт і запускається окремо

МОДУЛЬ 1

Платформа .NET та ASP.NET Core

1

- Що таке .NET, runtime і SDK
- Модель застосунку: хост → сервер → конвеєр
- Дві фази: builder → app
- Kestrel, моделі хостингу, середовища

Що таке .NET

.NET — платформа розробки: середовище виконання коду (runtime), стандартні бібліотеки та інструменти збірки. C# — лише одна з мов поверх неї.

Складова	Що це	Приклад
CLR (runtime)	віртуальна машина: JIT-компіляція IL, GC, потоки, безпека типів	coreclr
BCL	базові бібліотеки: колекції, LINQ, IO, HTTP, JSON, DI-абстракції	System.*
SDK	компілятор C# (Roslyn), MSBuild, CLI, шаблони, NuGet	dotnet build
ASP.NET Core	фреймворк для веб/API поверх .NET	Microsoft.AspNetCore.*
Мова	C# 14; F#; VB	record, pattern matching

- *.NET 10 — LTS-реліз (листопад 2025). Один і той самий runtime на Windows, Linux, macOS.*

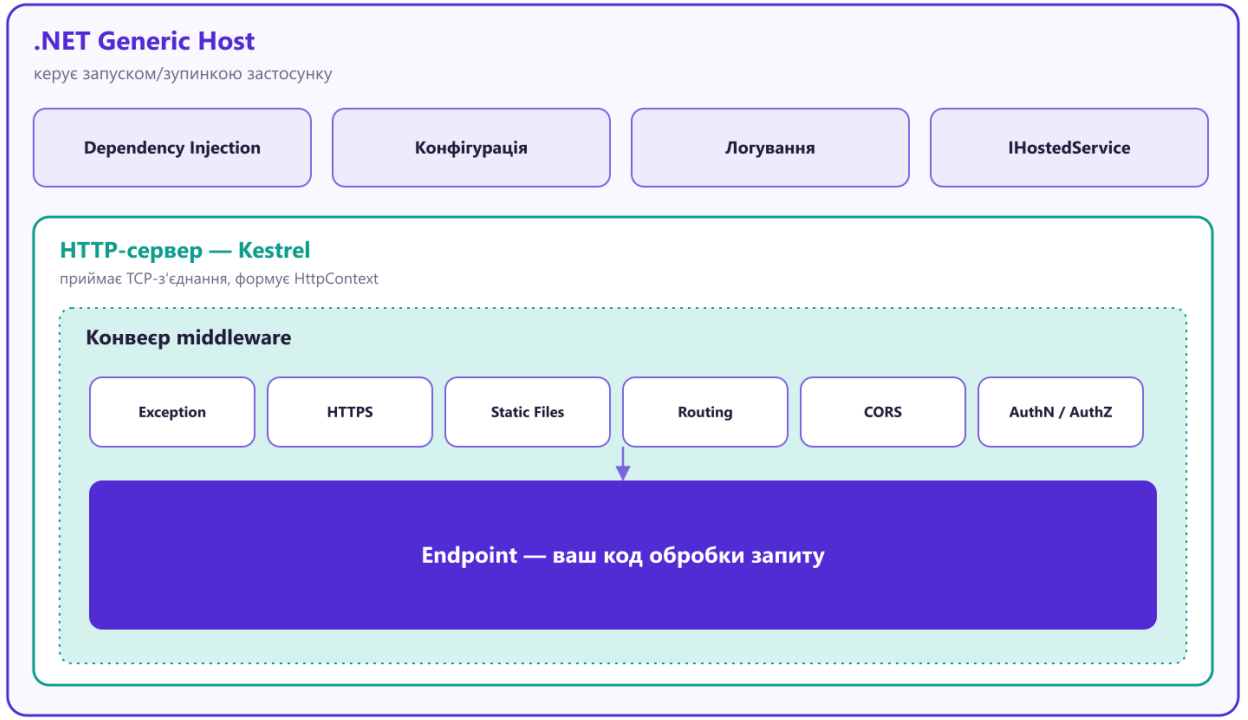
ASP.NET Core: принципи

ASP.NET Core — переписаний з нуля (2016) крос-платформний наступник класичного ASP.NET — модульний, швидкий, побудований навколо DI та конвеєра middleware.

1. **Крос-платформність** один код і один runtime на Windows, Linux, macOS; зручно для контейнерів
2. **Модульність** тонке ядро плюс NuGet-пакети — підключаєте лише те, що реально потрібно
3. **DI від першого дня** контейнер вбудований, і сам фреймворк побудований на впровадженні залежностей
4. **Конвеєр middleware** єдина проста абстракція обробки запиту замість старих HTTP-модулів і handlers
5. **Продуктивність** Kestrel — один із найшвидших HTTP-серверів; асинхронність усюди, мінімум алокацій
6. **Уніфікований хост** той самий Host для веб, фонових воркерів, gRPC і Blazor

Модель застосунку

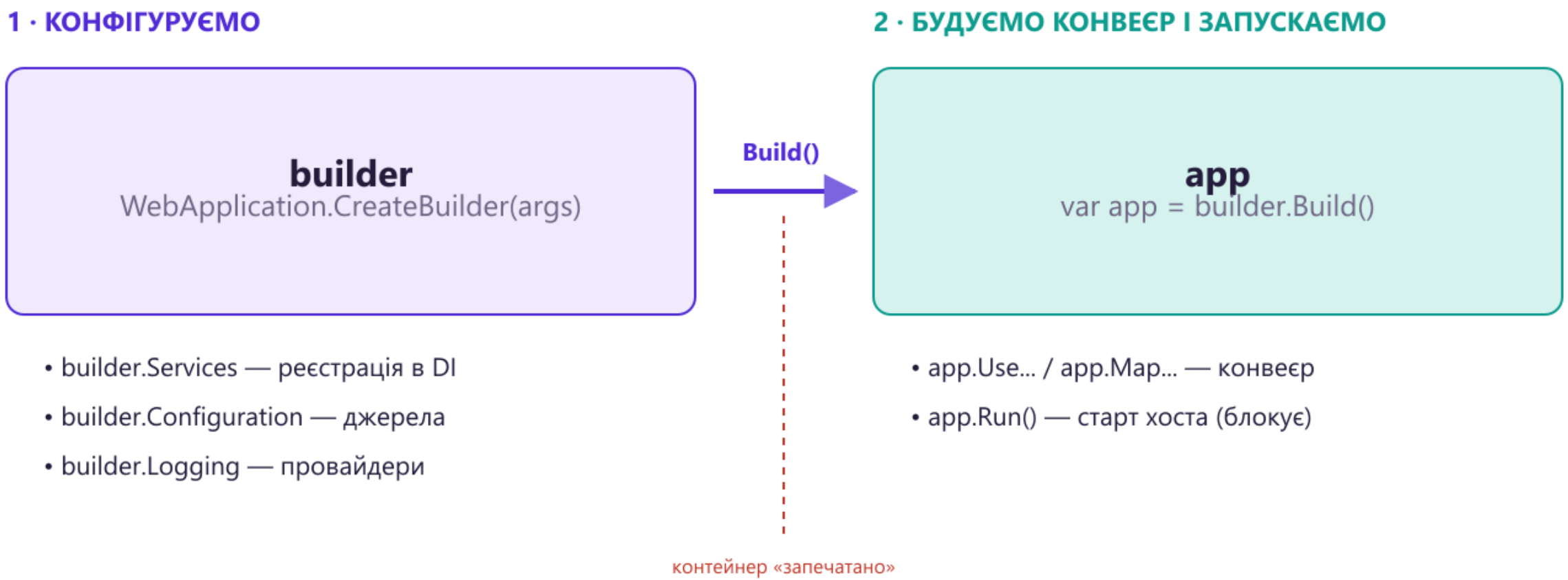
Host — загальний контейнер застосунку: керує запуском і зупинкою, тримає DI-контейнер, конфігурацію, логування та фонові сервіси. Той самий Host у веб, воркерів, gRPC.



- Host містить HTTP-сервер Kestrel (формує HttpContext), а той — конвеєр middleware, остання ланка якого ваш endpoint.

Дві фази: builder → app

CreateBuilder(args) конфігурує; Build() «запечатує» контейнер; далі — конвеєр і запуск



- Спроба додати сервіс після Build() — виняток. Склад контейнера має бути детермінованим.

Що CreateBuilder дає безкоштовно

Підсистема	Налаштовано за замовчуванням
Конфігурація	appsettings.json → appsettings.{Env}.json → User Secrets → env vars → CLI args
Логування	провайдери Console, Debug, EventSource
HTTP-сервер	Kestrel + інтеграція з IIS
DI-контейнер	Microsoft.Extensions.DependencyInjection
Середовище	IWebHostEnvironment зі змінної ASPNETCORE_ENVIRONMENT
DI-валідація	у Development перевіряються області (scopes)

- *Приклад 01 показує це мінімальним застосунком на кілька рядків.*

Життєвий цикл процесу

IHostApplicationLifetime дає три токени скасування. На сигнал SIGINT (Ctrl+C) або SIGTERM (від оркестратора) застосунок завершується не різко, а поетапно:

1. **1. ApplicationStopping** спрацьовує першим; сервер перестає приймати нові з'єднання
2. **2. Дренаж** поточні запити добігають до кінця — за замовчуванням до 30 секунд (ShutdownTimeout)
3. **3. IHostedService.StopAsync** зупиняються фонові сервіси у зворотному порядку реєстрації
4. **4. ApplicationStopped** спрацьовує наприкінці, процес виходить

- *Критично для Kubernetes: він надсилає SIGTERM перед видаленням поду; різкий обрив запитів = помилки в користувачів.*

HTTP-сервери

Сервер	Платформа	Коли обирають
Kestrel	крос-платформа	за замовчуванням; сам «дивиться в мережу» або за реверс-проксі
IIS (in-process)	Windows	AspNetCoreModuleV2 хостить застосунок усередині w3wp.exe
IIS (out-of-process)	Windows	IIS як проксі до Kestrel
HTTP.sys	Windows	потрібні Windows-автентифікація, port sharing, kernel-mode

- *Пріоритет джерел URL: Kestrel:Endpoints → ASPNETCORE_URLS → app.Urls → launchSettings.json (лише dotnet run).*

Середовища виконання

ASPNETCORE_ENVIRONMENT керує поведінкою застосунку

1. **Development** детальна сторінка помилки, Swagger/Scalar, seed-дані, гаряче перезавантаження
2. **Staging** максимально наближене до Production для фінальних перевірок
3. **Production** узагальнені помилки (ProblemDetails), увімкнений HSTS, мінімум діагностики
4. **Власне (напр. Testing)** env.IsEnvironment("Testing") — довільні розгалуження

- *Правило: детальні помилки, документація UI та тестові дані — ніколи не в Production.*

Модуль 1 — приклади

`01.Hosting.Minimal`

Мінімальний хост — builder → app → run, DI/конфіг/логування з коробки, graceful shutdown

`02.Hosting.Servers…`

Сервери та середовища — налаштування Kestrel, IWebHostEnvironment, appsettings.{Env}.json, dev-only endpoint

ФУНДАМЕНТ

HTTP — необхідний мінімум



- Структура запиту та відповіді
- Методи: safe / idempotent
- Родини статус-кодів
- HttpContext

Запит і відповідь

HTTP — протокол «запит — відповідь» без стану. У HTTP/1.1 текстовий поверх TCP; у HTTP/2 та HTTP/3 бінарний і мультиплексований, але семантика та сама. У ASP.NET Core усе це загорнуто в HttpContext.

HTTP-запит

- Рядок запиту: метод + шлях + версія
- напр. GET /products/42?expand=category HTTP/1.1
- Заголовки: Host, Accept, Authorization, Content-Type…
- Порожній рядок
- Тіло (необов'язкове): JSON, форма, файл

HTTP-відповідь

- Рядок статусу: версія + код + причина
- напр. HTTP/1.1 200 OK
- Заголовки: Content-Type, Location, ETag, Cache-Control…
- Порожній рядок
- Тіло: представлення ресурсу або опис помилки

Методи HTTP

Safe — не змінює стан; Idempotent — повтор дає той самий стан

Метод	Safe	Idempotent	Призначення
GET	так	так	прочитати ресурс або колекцію
HEAD	так	так	як GET, але без тіла (перевірити наявність / заголовки)
OPTIONS	так	так	які методи підтримує ресурс (CORS preflight)
POST	ні	ні	створити елемент у колекції; нестандартна дія
PUT	ні	так	повна заміна елемента за відомим URI
PATCH	ні	ні*	часткова зміна елемента
DELETE	ні	так	видалити елемент

- ** PATCH зазвичай не ідемпотентний. DELETE вже видаленого → 204/404, але не 500.*

Статус-коди

Родина	Значення	Ключові коди
1xx	інформаційні	100 Continue, 101 Switching Protocols
2xx	успіх	200 OK · 201 Created (+Location) · 204 No Content
3xx	перенаправлення / кеш	301/302 · 304 Not Modified
4xx	помилка клієнта	400 · 401 · 403 · 404 · 405 · 409 · 412 · 415 · 422 · 429
5xx	помилка сервера	500 · 502 · 503 · 504

- *401 «представся» ≠ 403 «тебе впізнали, але зась». 404 «немає» ≠ 405 «метод не для цього URI».*

HttpContext

HttpContext — єдиний об'єкт, у якому зібрано все про поточний запит; створюється сервером на кожен запит і живе рівно один запит.

1. **Request** метод, шлях, query, заголовки, тіло як потік, розібрані значення маршруту (RouteValues)
2. **Response** код статусу, заголовки, тіло; WriteAsync / WriteAsJsonAsync; прапорець HasStarted (після нього заголовки незмінні)
3. **User** ClaimsPrincipal — хто виконує запит; заповнюється після автентифікації
4. **Items** словник на час запиту — зручно передавати дані між middleware та endpoint
5. **RequestServices** DI-провайдер поточної області (scope), з якого резолвляться Scoped-сервіси

- *Stateless: сервер не тримає жодної сесії між запитами — кожен запит самодостатній, у ньому є все потрібне.*

МОДУЛЬ 2

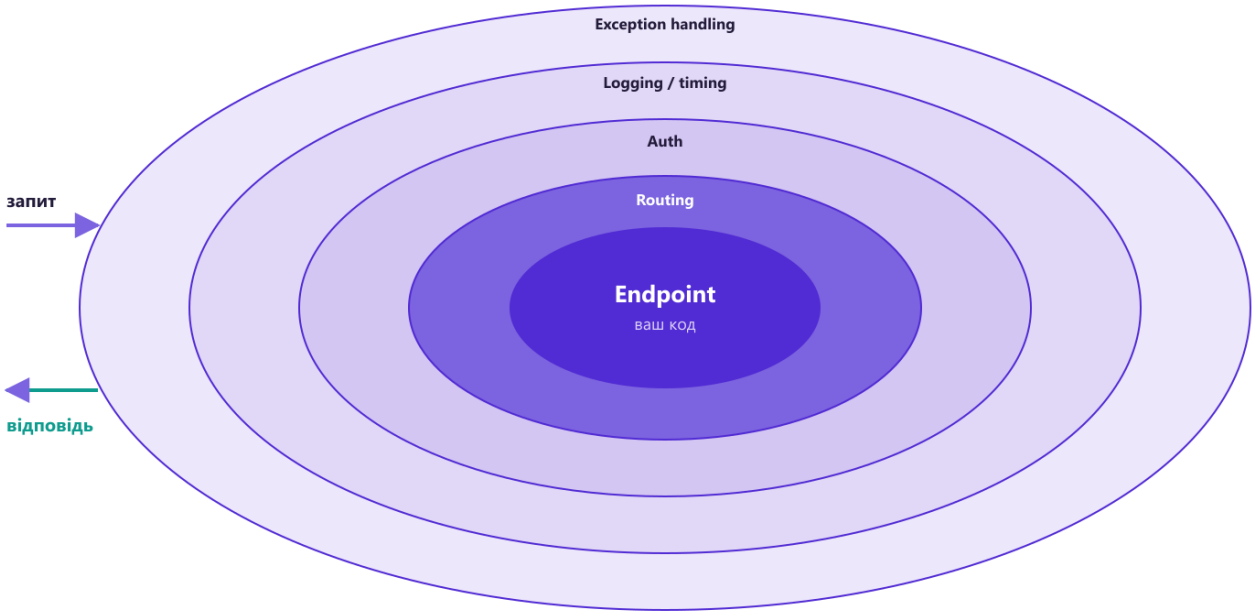
Конвеєр обробки запиту

2

- middleware, RequestDelegate, next
- обробка на вході / виході, short-circuit
- канонічний порядок
- розгалуження та наскрізні задачі

Головна ідея: конвеєр middleware

Middleware — компонент, що обробляє HTTP-запит і вирішує: щось із ним зробити та передати наступному компоненту, чи сформувати відповідь самому й далі не передавати. Запит проходить крізь усі middleware, доходить до endpoint і повертається назад тим самим шляхом.



код ДО await next() — на вході; код ПІСЛЯ — на виході (у зворотному порядку)

- Технічно кожен middleware — делегат, що отримує HttpContext і посилання на «наступний» (next).

Конвеєр = вкладені делегати

app.Use додає ланку; await next(context) провалюється глибше

```
app.Use(async (context, next) =>
{
    // код ДО  — обробка на вході
    await next(context);        // ← далі по конвеєру
    // код ПІСЛЯ — обробка на виході (стек розкручується назад)
});

app.Run(async context =>        // ТЕРМІНАЛЬНИЙ: next немає
{
    await context.Response.WriteAsync("кінець конвеєра");
});
```

- Use — звичайний middleware, може передати далі
- Run — термінальний, конвеєр завершується тут
- Не викликати next() → «коротке замикання» (auth, кеш, rate-limit)
- Map(path, …) — під-конвеєр за префіксом шляху

Три способи поводження middleware

Код, який ви написали ДО виклику await next, виконується на вході — у порядку реєстрації. Код ПІСЛЯ await next — на виході, у зворотному порядку, коли внутрішні шари вже відпрацювали.

1. **Наскрізний** просто await next(context) — нічого не робимо, керування йде далі по конвеєру

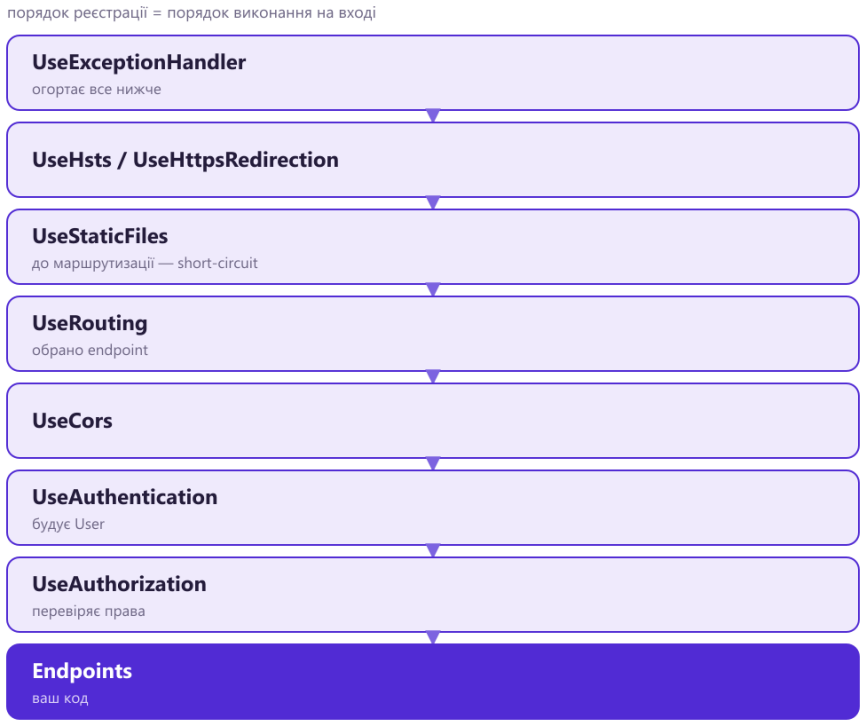
2. **Обробка входу або виходу** код до чи після next — так реалізують логування, вимірювання часу, додавання заголовків до відповіді

3. **Коротке замикання** не викликати next і самому сформувати відповідь — endpoint не виконається (auth, кеш, rate-limit)

- *Дописати заголовок з обробки на виході надійно можна лише через Response.OnStarting(callback) — після Response.HasStarted заголовки вже незмінні.*

Порядок middleware — критичний

Правило — порядок реєстрації middleware дорівнює порядку виконання на вході. Від нього критично залежить коректність — це найпоширеніша причина «магічних» багів у ASP.NET Core.



- UseCors після endpoint-ів → заголовки CORS не додаються. UseAuthorization до UseRouting → [Authorize] не бачить метаданих endpoint.

Власні middleware як класи

Інлайн-лямбда годиться для дрібниць. Коли middleware треба перевикористовувати або покрити тестами — його виносять у клас. Є два стилі, і головна різниця між ними — час життя екземпляра:

	За конвенцією	IMiddleware
Інтерфейс	немає	IMiddleware
Створення екземпляра	один раз на застосунок	на кожен запит
Реєстрація в DI	не потрібна	обов'язкова (AddScoped<T>)
Scoped-залежності	лише параметром InvokeAsync	можна в конструктор
Сигнатура	InvokeAsync(HttpContext)	InvokeAsync(HttpContext, RequestDelegate)

- *Патерн UseXxx() — метод-розширення, що ховає app.UseMiddleware<T>() за зрозумілою назвою (як UseRouting, UseCors).*

Розгалуження конвеєра

Іноді частині запитів потрібен інший набір middleware. «Термінальна» означає, що гілка завершує обробку сама й керування назад в основний конвеєр не повертається.

Метод	Умова	Термінальна?	Застосування
Map(path, …)	префікс шляху	так	під-застосунок: /admin, /health, вебхуки
MapWhen(pred, …)	будь-яка (заголовок, query)	так	гілка за нешляховою умовою
UseWhen(pred, …)	будь-яка	ні — повертається	умовний middleware: напр. auth лише для /api/*

- *Map обрізає префікс: усередині гілки Request.Path вже без нього, а Request.PathBase дорівнює префіксу.*

Наскрізні задачі (cross-cutting)

Наскрізна задача — функціональність, потрібна багатьом endpoint-ам одразу (обробка помилок, логування, CORS). У ASP.NET Core усе це — окремі middleware, які ставлять один раз у потрібному місці конвеєра.

1. **UseExceptionHandler** перехоплює необроблені винятки й перетворює на відповідь у форматі Problem Details (RFC 9457)
2. **UseStatusCodePages** додає тіло для «голих» статус-кодів, наприклад 404 без жодного тексту
3. **UseDefaultFiles / UseStaticFiles** віддають файли з wwwroot; ставляться до маршрутизації, щоб не робити зайвої роботи
4. **UseCors** дозволяє браузерному JavaScript звертатися з іншого origin; це послаблення обмеження браузера, а не захист сервера
5. **UseResponseCompression / Caching** стиснення та кешування відповідей

Модуль 2 — приклади

`03.Pipeline.FromScratch`	**Конвеєр з нуля** — Use/Run, next, обробка входу/виходу, коротке замикання, термінальний Run
`04.Pipeline.CustomMiddleware`	**Власні middleware** — конвенція vs IMiddleware, HttpContext.Items, патерн UseXxx()
`05.Pipeline.CrossCutting`	**Наскрізні задачі** — ProblemDetails, StatusCodePages, StaticFiles, CORS — і ПОРЯДОК
`06.Pipeline.Branching`	**Розгалуження** — Map / MapWhen / UseWhen, PathBase, повернення в основний конвеєр

МОДУЛЬ 3

Dependency Injection і конфігурація

3

- IoC та DI: навіщо
- Transient / Scoped / Singleton
- captive dependency
- патерн Options

Inversion of Control та DI

Dependency Injection — клас не створює свої залежності сам — він оголошує їх у параметрах конструктора, а спеціальний контейнер створює й «вручає» потрібні екземпляри при побудові об'єкта.

1. **Проблема** new SqlRepository() усередині класу — жорстка зв'язність, неможливо ані підмінити реалізацію, ані протестувати клас

2. **Рішення** клас приймає ICatalog у конструкторі; контейнер знаходить зареєстровану реалізацію й підставляє її

3. **Dependency Inversion** залежність від абстракції (інтерфейсу), а не від конкретного класу

4. **Вигоди** тестованість, легка заміна реалізації, єдине місце, де налаштовується час життя об'єктів

- *Увесь ASP.NET Core побудований на DI: контролери, middleware, Options, логери — усе резолвиться з контейнера.*

Реєстрація сервісів

IServiceCollection наповнюють до Build(); патерн Add{Feature}

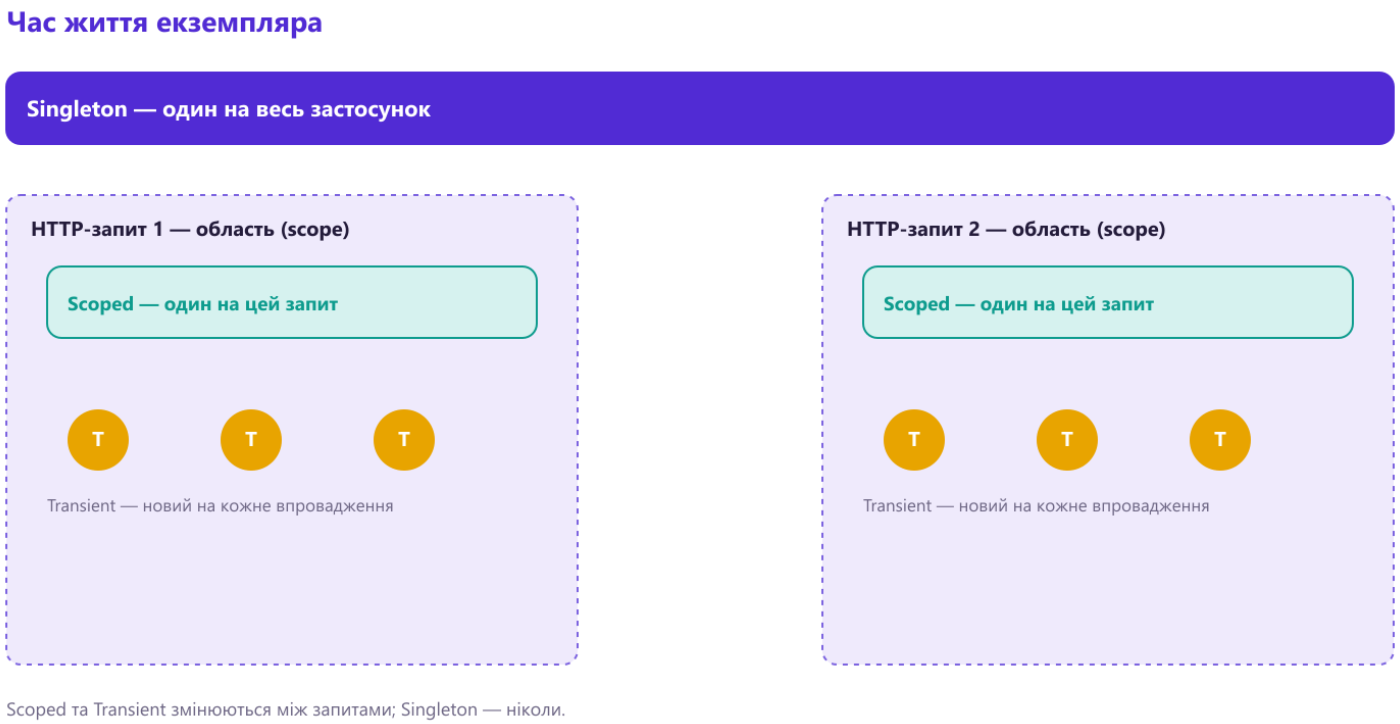
```
builder.Services.AddSingleton<ICatalog, InMemoryCatalog>();
builder.Services.AddScoped<IOrderService, OrderService>();
builder.Services.AddTransient<IEmailSender, SmtpSender>();

// бібліотека інкапсулює свої реєстрації в один метод:
public static IServiceCollection AddCatalog(this IServiceCollection
s)
{
    s.AddSingleton<ICatalog, InMemoryCatalog>();
    return s;
}
// builder.Services.AddCatalog();
```

- AddControllers(), AddAuthentication(), AddOpenApi() — той самий патерн
- Резолв: у параметри конструктора / дії
- [FromServices] з [ApiController] зазвичай не потрібен

Час життя сервісів

Час життя (lifetime) — правило, за яким контейнер вирішує, створити новий екземпляр сервісу чи віддати наявний. Transient — новий на кожне впровадження; Scoped — один на HTTP-запит; Singleton — один на весь застосунок.



Scoped та Transient змінюються між запитами; Singleton — ніколи.

- «Область» (scope) у веб-застосунку — це один HTTP-запит: ASP.NET Core створює її на кожен запит і звільняє в кінці.

Який час життя обрати

Час життя	Створюється	Типові кандидати
Transient	при кожному резолві	легкі stateless-хелпери, мапери, валідатори
Scoped	раз на HTTP-запит	DbContext, репозиторії, «одиниця роботи», контекст користувача
Singleton	раз на застосунок	кеш, конфіг, пул з'єднань, HttpClientFactory

- *За замовчуванням — Scoped для роботи з даними, Singleton для інфраструктури без стану запиту.*

Captive dependency (полонена залежність)

Captive dependency — коли Singleton приймає в конструктор Scoped- або Transient-сервіс. «Коротший» сервіс живе стільки ж, скільки Singleton — фактично стає Singleton, з усіма наслідками.

1. **Наслідки** витоки пам'яті, гонки даних, застарілі значення, ObjectDisposedException на вже звільненому DbContext

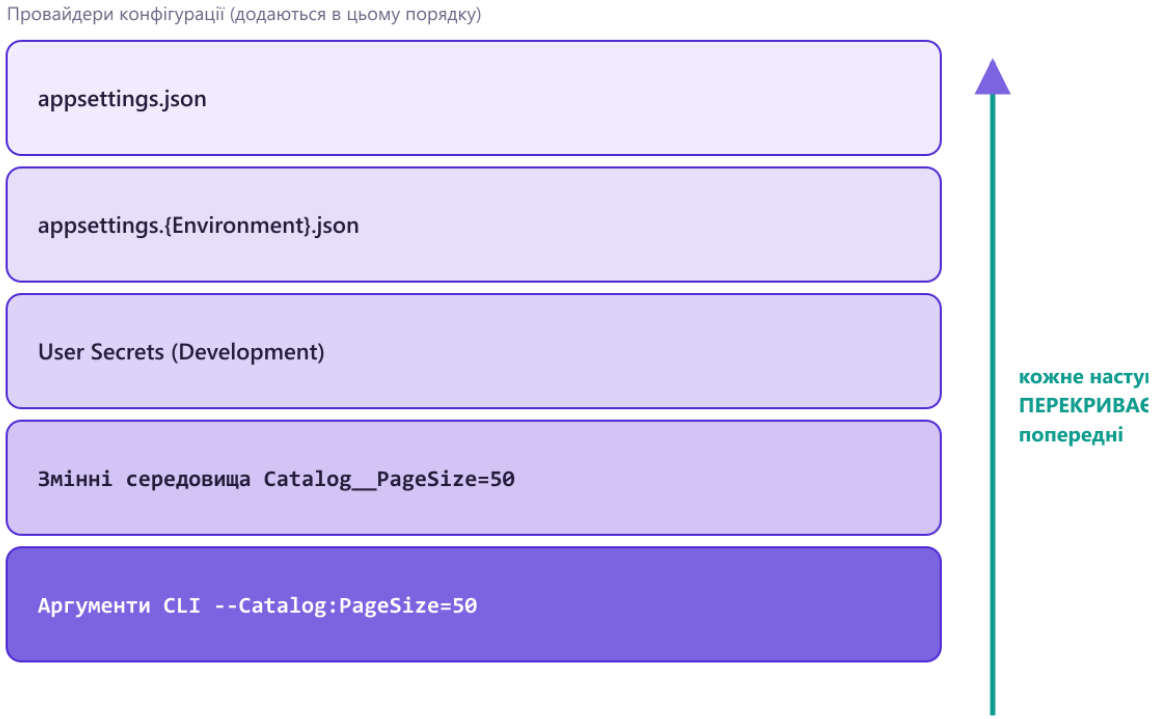
2. **Захист .NET** у Development хост валідує області на старті й кидає виняток «Cannot consume scoped service ... from singleton»

3. **Правильно** впровадити IServiceScopeFactory, у потрібний момент CreateScope() і резолвити сервіс уже з цієї області

4. **Окремий антипатерн** виклик BuildServiceProvider() у Program.cs — створює ДРУГИЙ контейнер з дублями синглтонів

Конфігурація: провайдери та пріоритет

Конфігурація — набір пар «ключ — значення», зібраний із кількох джерел (провайдерів). Порядок має значення: кожне наступне джерело перекриває попередні за однаковим ключем.



- ((IConfigurationRoot)config).GetDebugView() показує підсумкове значення кожного ключа й те, з якого джерела воно прийшло.

Патерн Options

Патерн Options — секцію конфігурації один раз прив'язують до типізованого класу — і далі по всьому коду впроваджують IOptions<T> замість того, щоб читати рядкові ключі вручну.

Тип	Час життя	Перечитує зміни	У middleware / singleton
IOptions<T>	Singleton	ні (обчислюється раз)	так
IOptionsSnapshot<T>	Scoped	так, раз на запит	ні
IOptionsMonitor<T>	Singleton	так, миттєво + OnChange	так

- *AddOptions<T>().Bind(section).ValidateDataAnnotations().ValidateOnStart() — падіння на старті, а не при першому доступі.*

Модуль 3 — приклади

`07.DependencyInjection…` **Час життя сервісів** — Transient/Scoped/Singleton на живому Guid; IServiceScopeFactory

`08.Configuration.Options` **Конфігурація та Options** — джерела й пріоритет, GetDebugView, IOptions/Snapshot/Monitor, валідація

МОДУЛЬ 4

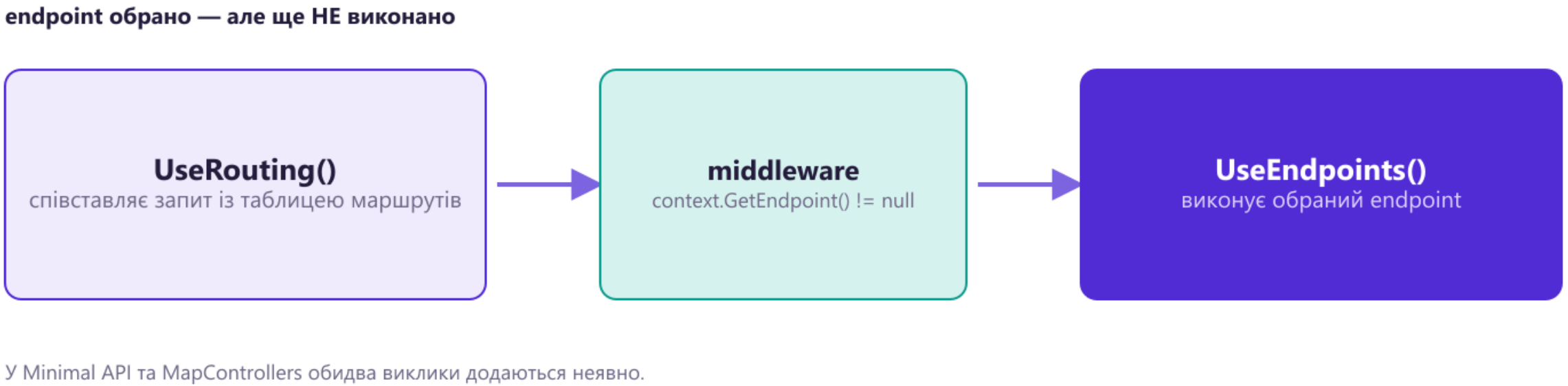
Маршрутизація

4

- endpoint routing — теж middleware
- дві фази: UseRouting / UseEndpoints
- шаблони, обмеження, пріоритет
- route values vs query string

Endpoint routing: дві фази

Маршрутизація (routing) — процес зіставлення URL запиту з конкретним обробником. У ASP.NET Core вона двоетапна й сама є middleware: спершу співставлення, потім — окремо — виконання.



У Minimal API та MapControllers обидва виклики додаються неявно.

- Між фазами можна вставити свій middleware: context.GetEndpoint() != null означає, що endpoint уже обрано, але ще не виконано.

Шаблони та обмеження маршрутів

Синтаксис	Значення
/products/{id}	обов'язковий параметр
{id:int}	обмеження типу — несумісний сегмент дає 404, а не 400
{page:int=1}	значення за замовчуванням
{slug?}	необов'язковий параметр
{**path}	catch-all — захоплює решту шляху разом зі слешами
:int :guid :bool :alpha :minlength(n) :range(a,b) :regex(...)	поширені обмеження

- *Обмеження — це фільтр СПІВСТАВЛЕННЯ, а не валідація. Конкретніший шаблон виграє: /products/featured перекриває /products/{id}.*

Route values vs query string

Дві частини URL мають різне призначення, і їх часто плутають. Шлях — це ідентичність ресурсу; query — параметри операції над ним.

1. **Шлях ідентифікує РЕСУРС** /products/42, /categories/laptops, /products/42/reviews — стабільна адреса конкретної сутності

2. **Query — параметри операції над колекцією** ?q=hub&page=2&sort=price — фільтрування, сортування, пагінація вибірки

3. **Правило** не кладіть у шлях те, що не є частиною ідентичності ресурсу

4. **Генерація URL** не склеюйте адреси рядками — LinkGenerator / IUrlHelper будують їх за ІМЕНЕМ маршруту ([HttpGet("…", Name = "…")]), враховуючи PathBase й кодування

Модуль 4 — приклад

`09.Routing.Endpoints`

Endpoint routing

constraints, значення за замовчуванням, catch-all, пріоритет, LinkGenerator, middleware між фазами

МОДУЛЬ 5

Побудова API: контролери

5

- AddControllers / MapControllers
- [Route], [HttpGet…], ActionResult<T>
- прив'язка параметрів
- валідація: авто-400 + ModelState

Контролер = ресурс

Контролер — клас, що групує споріднені endpoint-и («дії») одного ресурсу; [Route] — базовий шлях, атрибути дій — HTTP-метод і сегмент.

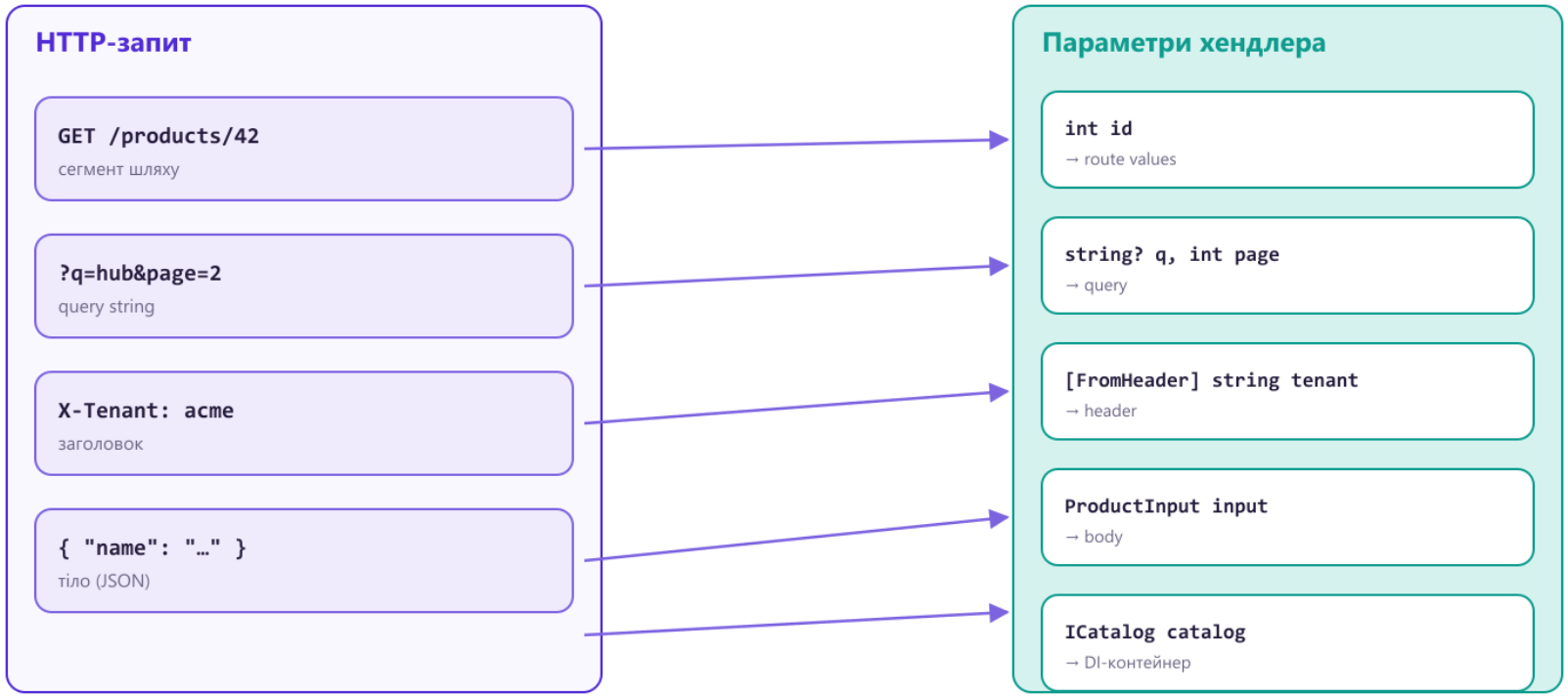
```
[ApiController]
[Route("products")]
public sealed class ProductsController(ICatalog c) : ControllerBase
{
    [HttpGet("{id:int}")]                  // 200 або 404
    public ActionResult<Product> GetById(int id)
        => c.FindProduct(id) is { } p ? p : NotFound();

    [HttpPost]                             // 201 + Location
    public ActionResult<Product> Create(ProductInput dto)
    {
        var p = c.Add(dto);
        return CreatedAtAction(nameof(GetById), new { id = p.Id },
p);
    }
}
```

- AddControllers() — інфраструктура MVC; MapControllers() — дії в таблицю маршрутів
- ActionResult<T> — 200 з T АБО інший результат (NotFound(), BadRequest())
- CreatedAtAction(...) — 201 + Location за іменем дії, не конкатенацією
- Залежності — у первинний конструктор класу

Прив'язка параметрів

Прив'язка (model binding) — автоматичне заповнення параметрів дії значеннями із запиту. Фреймворк сам вирішує, звідки брати кожен параметр, за його типом та ім'ям.



- Тіло запиту можна прочитати лише ОДИН раз і лише для ОДНОГО параметра.

Джерела прив'язки

Джерело	Коли за замовчуванням	Явний атрибут
Route values	ім'я = сегмент {…} у шаблоні	[FromRoute]
Query string	простий тип, не в маршруті	[FromQuery]
Тіло (JSON)	складний тип (з [ApiController])	[FromBody]
Заголовок	—	[FromHeader(Name=…)]
DI-контейнер	тип зареєстровано як сервіс	[FromServices]
Спеціальні	HttpContext, ClaimsPrincipal, CancellationToken	—

▸ *Одну модель можна зібрати з різних джерел, поставивши [FromRoute]/[FromQuery] на її властивості. Власні прості типи: реалізуйте IParsable<T> (TryParse).*

Валідація вводу

Валідація — це перевірка, що зв'язані дані мають сенс (обов'язкові поля заповнені, число в діапазоні). Дві стратегії, які доповнюють одна одну:

1. **Авто-400 від [ApiController]** моделі з атрибутами DataAnnotations (Required, Range, StringLength) перевіряються ДО тіла дії; помилка → 400 application/problem+json з полем errors

2. **Крос-польові правила — ModelState** ModelState.AddModelError(поле, текст) + return ValidationProblem(ModelState) — для правил, яких нема в атрибутах, і звірки з базою

3. **Прив'язка ≠ валідація** якщо int узагалі не зв'язався (отримав "abc") — це помилка ПРИВ'ЯЗКИ, вона дає 400 ще раніше, з іншим тілом

- *Коди CRUD: список/читання 200 · створення 201 + Location · заміна 200/204 · видалення 204 · немає ресурсу 404.*

Модуль 5 — приклади

`10.Mvc.Crud` **Повний CRUD** — [Route]/[HttpGet…], ActionResult<T>, CreatedAtAction, коди відповідей, ідемпотентність

`11.Mvc.Binding…` **Прив'язка та валідація** — [FromRoute]/[FromQuery]/[FromHeader], авто-400, ModelState

МОДУЛЬ 6

MVC глибше: [ApiController] і фільтри

6

- що саме вмикає [ApiController]
- ActionResult<T>, ModelState, ProblemDetails
- 5 типів фільтрів і їхній порядок
- фільтр vs middleware

Інфраструктура MVC

MVC (для API) — модель роботи, де обробники згруповані в класи-контролери, а фреймворк бере на себе активацію, model binding, форматери й ApiExplorer. Приклади 07–11 уже на контролерах — тут розбираємо механізми детальніше.

1. **Коли особливо доречно** багато споріднених endpoint-ів зі спільною поведінкою, потрібні фільтри й конвенції, успадкування, великі команди

2. **AddControllers()** реєструє інфраструктуру MVC: активацію контролерів, model binding, форматери, ApiExplorer

3. **MapControllers()** додає всі знайдені дії контролерів до таблиці маршрутів

4. **ControllerBase** для API (без підтримки Razor View); повний Controller — лише коли рендеряться HTML-уявлення

[ApiController] та ActionResult<T>

[ApiController] вмикає

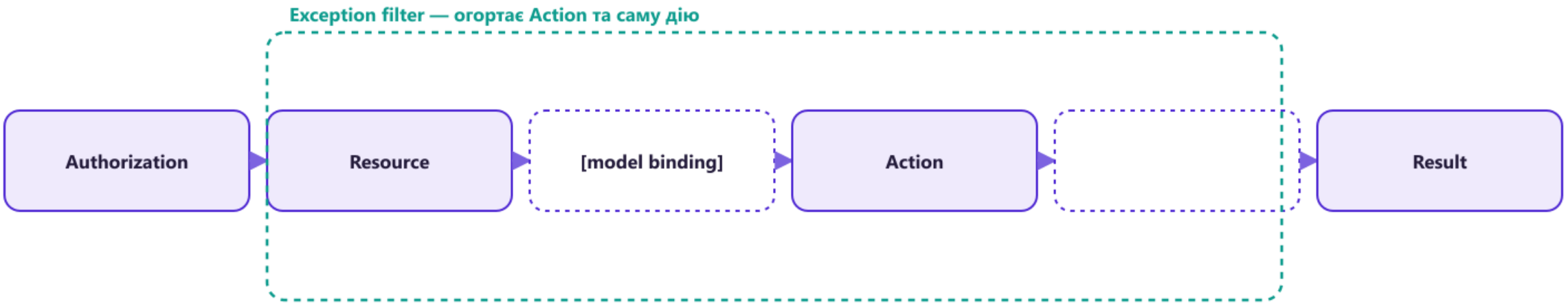
- авто-400 з ValidationProblemDetails при невалідній ModelState (до входу в метод)
- інференс джерела прив'язки (складний тип → тіло)
- ProblemDetails для проблемних статус-кодів
- обов'язкову атрибутну маршрутизацію

Типи повернення

- ActionResult<T> — 200 з T АБО інший результат (NotFound(), BadRequest())
- IActionResult — лише результати, без «типу успіху» для OpenAPI
- T — завжди 200; помилки лише через винятки/фільтри
- CreatedAtAction(...) — 201 + Location за іменем дії

Фільтри MVC: конвеєр усередині конвеєра

Фільтр — спеціалізований middleware для MVC, який виконується навколо дії контролера. На відміну від звичайного middleware, він знає про обрану дію, її аргументи та результат.



Фільтр, на відміну від middleware, знає про обрану дію, її аргументи та результат.

- П'ять типів у порядку: Authorization → Resource → [binding] → Action → [ДІЯ] → Result; Exception огортає Action та дію.

Фільтр vs middleware

	Middleware	Фільтр
Область	увесь конвеєр	лише MVC-запити
Знає про дію/модель	ні	так (ActionDescriptor, ActionArguments, Result)
Порядок	порядок реєстрації	тип + Order + scope (global/controller/action)
Типи	один	Authorization / Resource / Action / Exception / Result

- *Наскрізне й не-MVC (логування всіх запитів, стиснення) → middleware. Специфічне для дій (конверт відповіді, валідація аргументів) → фільтр.*

Є ще Minimal API

Minimal API — компактніший стиль: endpoint — це лямбда або метод, переданий у app.MapGet / MapGroup, без класу-контролера. Той самий конвеєр і DI. Цей курс — на контролерах; знати про Minimal API варто, бо його багато в документації й шаблонах.

1. **Форма** app.MapGet("/products/{id}", (int id, ICatalog c) => …); MapGroup для спільного префікса
2. **Результати** Results / TypedResults; union-тип Results<Ok<T>, NotFound> дає OpenAPI всі коди без атрибутів
3. **Фільтри** AddEndpointFilter — «middleware рівня endpoint» замість 5 типів MVC-фільтрів
4. **Коли обирають** малі та середні API, мікросервіси, мінімум церемоній; можна поєднувати з контролерами в одному застосунку

Модуль 6 — приклади

`12.Mvc.Controllers`

Контролери для Web API — [ApiController], атрибутна маршрутизація, ActionResult<T>, ModelState, CreatedAtAction

`13.Mvc.Filters`

Фільтри MVC — Resource/Action/Result/Exception, порядок виконання, відмінність від middleware

МОДУЛЬ 7

REST-архітектура

7

- Roy Fielding — стиль, а не протокол
- 6 обмежень; ресурс / представлення
- дієслова, коди, ідемпотентність
- модель зрілості Річардсона

Що таке REST

REST — REpresentational State Transfer — архітектурний СТИЛЬ (Рой Філдінг, дисертація 2000, розділ 5): набір обмежень на архітектуру розподіленої системи. HTTP — найпоширеніше, але не єдине його втілення.

1. **Це стиль, а не протокол** REST не диктує формат повідомлень — він задає принципи: ресурси, єдиний інтерфейс, відсутність стану
2. **Ресурс** будь-що, що можна назвати — товар, замовлення, звіт; має стабільний URI-ідентифікатор
3. **Представлення** клієнт і сервер обмінюються знімками стану ресурсу у форматі JSON чи XML, а не самими внутрішніми об'єктами
4. **Стан застосунку** рухається між клієнтом і сервером через ці представлення та посилання в них

6 обмежень REST

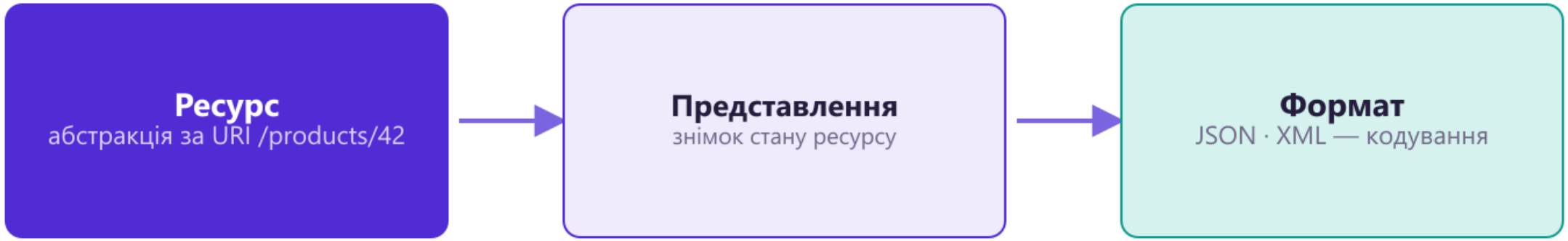
Архітектуру називають RESTful, коли вона задовольняє ці обмеження. Найважливіші для практики — «без стану» та «єдиний інтерфейс».

Обмеження	Що означає	Вигода
Клієнт–сервер	розділення відповідальності (UI ↔ дані)	незалежна еволюція
Без стану (stateless)	кожен запит містить усе потрібне; сервер не тримає сесію	масштабованість, надійність
Кешованість	відповідь позначає, чи можна її кешувати	продуктивність, менше навантаження
Єдиний інтерфейс	ресурси + стандартні дієслова + представлення (+ HATEOAS)	простота, слабка зв'язність
Багатошаровість	проксі, шлюзи, балансувальники прозорі для клієнта	гнучкість інфраструктури
Code-on-demand	сервер може віддати виконуваний код (необов'язково)	розширюваність клієнта

Ресурс ≠ представлення ≠ формат

Представлення (representation) — конкретний знімок стану ресурсу в певний момент. Ресурс — абстракція за URI; представлення — його поточний вміст; формат (JSON, XML) — лише кодування цього представлення.

Клієнт обмінюється з сервером ПРЕДСТАВЛЕННЯМИ, а не внутрішніми об'єктами



Accept: application/xml → той самий ресурс, інше кодуванн

- Один ресурс може мати кілька представлень і кілька форматів; клієнт обирає бажаний формат заголовком Accept.

Ідемпотентність — детально

Ідемпотентність — властивість методу: повторний однаковий запит не змінює стан системи ДОДАТКОВО. Практична цінність — ідемпотентні запити можна безпечно повторювати при обриві мережі.

1. **GET / HEAD / OPTIONS** і safe, і idempotent — читання взагалі нічого не змінює

2. **PUT** idempotent — надіслати те саме повне представлення двічі = двічі привести ресурс до того самого стану

3. **DELETE** idempotent — видалення вже видаленого повертає 204 або 404, бо клієнт хотів «щоб ресурсу не було»

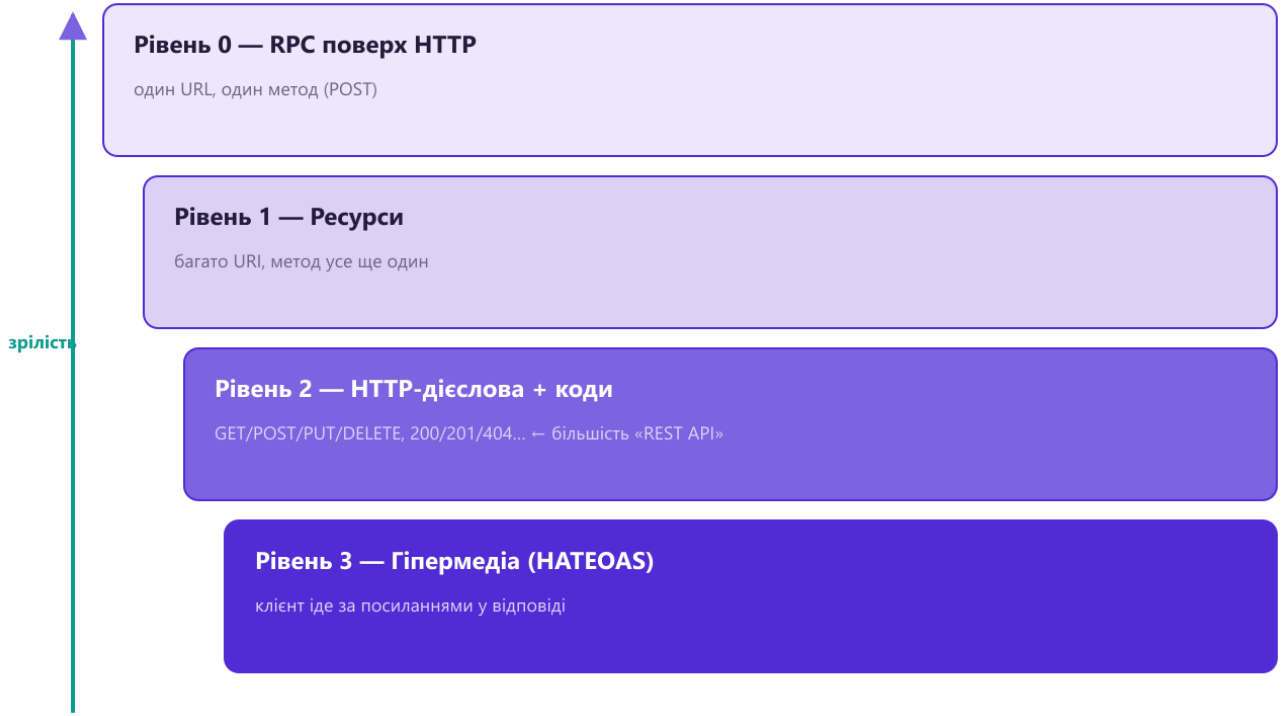
4. **POST** НЕ idempotent — кожен виклик створює новий ресурс; для захисту від дублів — заголовок Idempotency-Key

Коди відповідей для CRUD API

Код	Коли
200 / 201 / 204	успіх з тілом / створено (+Location) / успіх без тіла
400 Bad Request	синтаксично невірний запит або помилка прив'язки
401 / 403	не автентифікований / немає прав
404 / 405	ресурсу немає / метод не підтримується для URI (є заголовок Allow)
409 Conflict	конфлікт стану (дубль ключа, застаріла версія)
412 / 428	передумова не виконана / передумова обов'язкова (ETag)
415 / 422	непідтримуваний Content-Type / семантично невірні дані
429 / 500 / 503	забагато запитів / помилка сервера / сервіс недоступний

Модель зрілості Річардсона

Модель Річардсона — шкала з чотирьох рівнів, що показує, наскільки API використовує можливості HTTP — від RPC поверх одного URL до повноцінного гіпермедіа.



- Більшість промислових API живуть на рівні 2 — і це нормально. Рівень 3 корисний для складних робочих процесів.

Дизайн URI

URI — це ім'я ресурсу, і його форма має бути передбачуваною. Кілька практичних правил:

1. **Іменник, а не дієслово** /products/42 — правильно; /getProduct?id=42 — ні (дія закодована в методі HTTP)
2. **Колекція — у множині** /products — колекція; /products/42 — елемент; /products/42/reviews — відношення через вкладеність
3. **Дії поза CRUD — під-ресурс стану** POST /orders/42/cancellation читабельніше й «ресурсніше», ніж POST /orders/42/cancel
4. **Фільтри й пагінація — у query** ?category=laptops&minPrice=100&sort=-price&page=2 — не в шляху
5. **Стабільність** URI ресурсу не повинен «стрибати» між версіями — версіонуйте контракт відповіді, а не ідентичність

Модуль 7 — приклад

`14.Rest.ResourcesAndVerbs`

Ресурси та дієслова

семантика GET/POST/PUT/PATCH/DELETE, ідемпотентність, коди відповідей, Location, 405/415

МОДУЛЬ 7 · далі

REST: представлення, колекції, версії

7

- content negotiation, ETag, умовні запити
- PATCH: JSON Patch vs Merge Patch
- пагінація, Problem Details
- версіонування, гіпермедіа

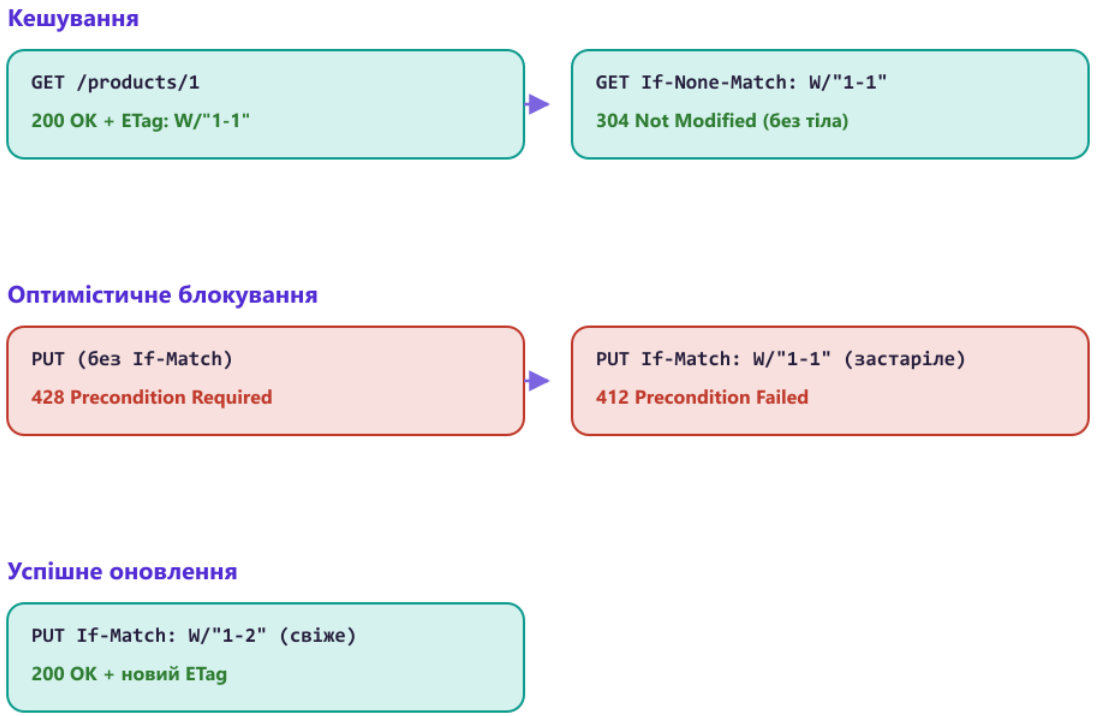
Content negotiation

Content negotiation — узгодження формату: клієнт заголовком Accept каже, у якому вигляді хоче відповідь, а сервер обирає відповідний форматер (formatter) і повертає той самий ресурс у потрібному кодуванні.

1. **Accept: application/xml** сервер обирає output formatter; MVC із коробки вміє JSON, XML підключається окремо
2. **406 Not Acceptable** якщо жоден форматер не підходить і ввімкнено ReturnHttpNotAcceptable — замість «мовчки JSON»
3. **Content-Type + [Consumes]** аналогічно для тіла запиту працює input formatter
4. **System.Text.Json** AddJsonOptions — іменування властивостей, ігнорування null, формат дат

Умовні запити: ETag

ETag — «версійний штамп» представлення, який сервер повертає в заголовку. Клієнт потім використовує його: If-None-Match — для кешу (304), If-Match — щоб не затерти чужі зміни (412).



- ETag виду W/"..." — слабкий (семантична еквівалентність); "..." — сильний (побайтова).

Оптимістичне блокування

Оптимістичне блокування — спосіб уникнути «lost update», коли двоє клієнтів редагують ту саму версію. Ресурс не блокують — просто перевіряють версію (ETag) у момент запису й відхиляють запит зі старою версією.

1. **Проблема без ETag** клієнт A і B читають v1; A зберігає свою зміну; B зберігає поверх — зміни A мовчки зникли, ніхто не дізнався

2. **З ETag** GET повертає ETag; PUT або PATCH клієнт зобов'язаний надіслати з If-Match; версія не збіглася → 412, треба перечитати й повторити

3. **428 Precondition Required** сервер може вимагати заголовок If-Match для будь-якої зміни

PATCH: два формати

PATCH описує ЧАСТКОВУ зміну ресурсу. Стандарт не один — є два несумісні формати тіла, і клієнт має вказати, який використовує, через Content-Type.

	JSON Patch (RFC 6902)	JSON Merge Patch (RFC 7386)
Content-Type	application/json-patch+json	application/merge-patch+json
Тіло	список операцій	частковий об'єкт
Приклад	[{"op":"replace","path":"/price","value":42}]	{ "price": 42 }
Операції	add/remove/replace/move/copy/test	лише присвоєння; null = видалити
Масиви	точкові зміни за індексом	заміна цілком

- *.NET 10: пакет Microsoft.AspNetCore.JsonPatch.SystemTextJson — без Newtonsoft.*

Колекції: пагінація

Пагінація — віддавання колекції частинами замість усього масиву одразу. Два підходи: за зсувом (номер сторінки) або за курсором (від останнього побаченого елемента).

	Offset (page/pageSize)	Cursor (keyset)
Просто реалізувати	так	ні
Стабільність при вставках	ні (зсув сторінок)	так
«Перейти на сторінку N»	так	ні
Великі дані / нескінченний скрол	повільно (OFFSET)	швидко

- *Завжди задавайте max pageSize. Сортування — тільки за білим списком полів, ніколи не підставляйте поле з query напряму.*

Problem Details (RFC 9457)

Problem Details — стандартний формат тіла помилки з Content-Type application/problem+json. Один формат на весь API — і клієнт пише обробку помилок лише раз.

```
{
  "type": "https://tools.ietf.org/html/rfc9110#section-15.5.1",
  "title": "One or more validation errors occurred.",
  "status": 400,
  "instance": "/products",
  "errors": {
    "name":  ["The Name field is required."],
    "price": ["The field Price must be between 0.01 and 100000."]
  },
  "traceId": "00-2f8b...-01"
}
```

- AddProblemDetails() — єдиний формат для 4xx/5xx фреймворку
- Results.ValidationProblem(errors) / Results.Problem(...)
- Один формат на весь API — клієнт пише обробку помилок раз
- CustomizeProblemDetails — додати traceId, instance

Версіонування API

Версіонування — підтримка кількох версій контракту API одночасно. Потрібне, бо публічний API — це контракт: breaking change замість «зламати всіх» вводять новою версією, стару якийсь час підтримують.

Спосіб	Приклад	Плюс / мінус
Сегмент URL	/v2/products	видно, кешується / URI «стрибає»
Query string	/products?api-version=2.0	не чіпає шлях / легко забути
Заголовок	X-Api-Version: 2.0	URI стабільний / «невидимо»
Media type	Accept: application/json;v=2.0	найбільш «RESTful» / складно для клієнтів

- *ReportApiVersions → заголовки api-supported-versions / api-deprecated-versions. Адитивні зміни (нове необов'язкове поле) — не breaking.*

Гіпермедіа (HATEOAS)

HATEOAS — Hypermedia As The Engine Of Application State — рівень 3 Річардсона. Клієнт рухається по API не за «зашитими» URL, а за посиланнями (_links), які сервер повертає у відповіді.

1. **Стан → дії** набір доступних посилань залежить від СТАНУ ресурсу: порожній кошик не має посилання «оформити», непорожній — має
2. **Вигода** менша зв'язність; дозволені переходи визначає сервер в одному місці; API можна еволюціонувати без оновлення клієнтів
3. **Формати** HAL (_links, _embedded), JSON:API, Siren, Hydra
4. **Прагматично** навіть без повного HATEOAS корисно додавати self-посилання та next/prev для колекцій — це дешево

Модуль 7 · далі — приклади

`15.Rest.Representations` — **Представлення** — content negotiation (JSON/XML), ETag, If-None-Match/If-Match, JSON Patch

`16.Rest.CollectionsAndErrors` — **Колекції та помилки** — пагінація, фільтр, конверт відповіді, X-Total-Count, ProblemDetails

`17.Rest.Versioning` — **Версіонування** — 4 способи вказати версію, v1/v2 одночасно, deprecation

`18.Rest.Hypermedia` — **Гіпермедіа** — _links, стан ресурсу → доступні дії, модель Річардсона

МОДУЛЬ 8

Документація та тестування

8

- OpenAPI: що це, історія Swagger
- .NET 10: AddOpenApi / MapOpenApi
- Swagger UI · Scalar · Redoc
- піраміда тестів; WebApplicationFactory

OpenAPI

OpenAPI — машиночитний опис HTTP API у форматі JSON/YAML — шляхи, методи, параметри, схеми, коди, безпека. Раніше називався Swagger Specification.

1. **Що з нього роблять** інтерактивну документацію, клієнтські SDK різними мовами, моки, контрактні тести в CI, імпорт у Postman чи API-шлюз

2. **.NET 10** AddOpenApi() + MapOpenApi() генерують /openapi/v1.json вбудовано, без жодних сторонніх пакетів

3. **Джерело правди** документ один; переглядачів (UI) поверх нього може бути скільки завгодно й одночасно

4. **Метадані** беруться з таблиці маршрутів, типів результату, атрибутів, XML-коментарів та трансформерів

Джерела метаданих для OpenAPI

Що в документі	Звідки
шляхи, методи, параметри	таблиця маршрутів
схеми запиту/відповіді	типи параметрів і результату (ActionResult<T>)
коди відповідей	ActionResult<T> + [ProducesResponseType<T>(код)]
summary / description	XML-коментарі ///, [EndpointSummary] / [EndpointDescription]
теги (групування)	[Tags("…")] на контролері
приклади, Info, security	трансформери: AddDocumentTransformer / AddSchemaTransformer

Переглядачі: Swagger UI · Scalar · Redoc

	Swagger UI	Scalar	Redoc
Пакет	Swashbuckle…SwaggerUI	Scalar.AspNetCore	статикою
«Try it out»	так	так	ні (лише читання)
.NET-шаблони	до .NET 8	.NET 9+	—
Особливість	класика, впізнаваний	сучасний, генерує сніпети коду	гарний для публічних API

- *У курсі приклад 19 підключає Swagger UI (/swagger) і Scalar (/scalar) паралельно — один документ, два UI.*

Тестування: піраміда

WebApplicationFactory<T> — піднімає РЕАЛЬНИЙ застосунок у пам'яті — весь конвеєр, DI, маршрутизацію, серіалізацію — і дає HttpClient, який звертається до нього без справжньої мережі.



- Це найдешевший спосіб перевірити endpoint-и «як бачить клієнт». TEntryPoint — клас Program цільового проєкту (public partial class Program).

Модуль 8 — приклади

`19.OpenApi.Swagger` **OpenAPI + два UI** — AddOpenApi/MapOpenApi, трансформери, Swagger UI та Scalar паралельно

`20.Testing` **Інтеграційні тести** — WebApplicationFactory, HttpClient без мережі, перевірка кодів і Location

МОДУЛЬ 9

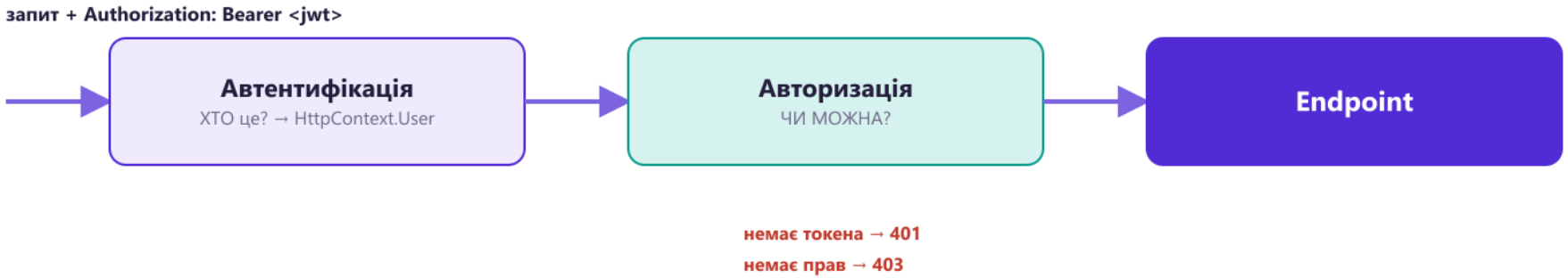
Безпека (базово)

9

- автентифікація vs авторизація
- JWT: структура, Bearer
- [Authorize], політики, ролі, claims
- місце auth у конвеєрі

Автентифікація vs авторизація

AuthN vs AuthZ — Автентифікація (AuthN) відповідає «ХТО це?» — встановлює особу й заповнює HttpContext.User. Авторизація (AuthZ) відповідає «ЧИ МОЖНА?» — перевіряє права вже встановленої особи. Це дві різні речі.



JSON Web Token



Сервер не зберігає токен — перевіряє підпис і термін. Тому: тільки HTTPS, короткий exp.

- 401 «ти не представився» (немає/невалідний токен) ≠ 403 «я тебе впізнав, але доступу немає».

JWT та схема Bearer

JWT — JSON Web Token — самодостатній токен із трьох частин через крапку (base64url): header, payload (claims), signature. Клієнт надсилає його в заголовку Authorization: Bearer <jwt>.

1. **Claims** у payload: sub (ідентифікатор користувача), role, exp (час протухання), плюс власні

2. **Підпис** HMAC-SHA256 симетричним ключем або RSA/ECDSA асиметрично

3. **Сервер не зберігає токен** лише перевіряє підпис і термін; що саме перевіряти — задається через TokenValidationParameters

4. **Наслідок** токен = перепустка на пред'явника → працювати тільки по HTTPS, короткий exp, для сесій — refresh-токени

Авторизація: механізми

Механізм	Приклад	Коли
[Authorize]	будь-хто автентифікований	приватний ресурс
[Authorize(Roles="admin")] / RequireRole	claim role=admin	грубий поділ
RequireClaim("scope", "…")	довільний claim	фічі, тарифи, OAuth scopes
політика + RequireAssertion	age >= 18, «власник ресурсу»	складні правила

- *Порядок у конвеєрі: UseRouting → UseAuthentication → UseAuthorization → endpoints. Політики іменують і перевикористовують.*

Модуль 9 — приклад

`21.Security.AuthBasics`

Основи безпеки

JWT bearer, видача токена, [Authorize], політика ролі, місце auth у конвеєрі

ПІДСУМОК

Повна картина



- карта курсу — 21 приклад
- життєвий цикл запиту
- що вивчати далі

Життєвий цикл запиту — велика картина



- Kestrel формує HttpContext → конвеєр middleware → routing обирає endpoint → endpoint виконується → відповідь назад тим самим конвеєром.

Карта курсу

Модуль	Приклади	Ключові концепції
Хостинг	01–02	Host, builder→app, Kestrel, середовища, graceful shutdown
Конвеєр	03–06	middleware, next, порядок, Map/UseWhen, ProblemDetails, CORS
DI + конфіг	07–08	Transient/Scoped/Singleton, captive dependency, Options
Маршрутизація	09	endpoint routing, constraints, пріоритет, LinkGenerator
Web API	10–11	контролери, ActionResult<T>, прив'язка, ModelState
MVC глибше	12–13	[ApiController], ProblemDetails, 5 фільтрів
REST	14–18	дієслова, коди, ETag, PATCH, пагінація, версії, HATEOAS
Доку + тести	19–20	OpenAPI, Swagger/Scalar, WebApplicationFactory
Безпека	21	AuthN/AuthZ, JWT, [Authorize], політики

Що вивчати далі

1. **Доступ до даних** EntityFramework Core, Dapper; міграції, транзакції, патерн репозиторію
2. **Реалтайм і RPC** SignalR (WebSockets), gRPC, gRPC-Web
3. **UI** Blazor (Server / WebAssembly / United), Razor Pages, MVC Views
4. **Продакшн** деплой (Docker, Kubernetes, Azure), health checks, HTTPS/сертифікати
5. **Спостережуваність** структуроване логування (Serilog), OpenTelemetry, метрики, розподілене трасування
6. **Стійкість** Polly (retry, circuit breaker), rate limiting, кешування (Redis, HybridCache)

Посилання

Офіційна документація

- learn.microsoft.com/aspnet/core — Fundamentals
- Middleware · Dependency injection · Configuration
- Routing · Web API with controllers · Model binding & validation
- OpenAPI · Testing · Security / Authentication

Специфікації та посібники

- RFC 9110 — HTTP Semantics (методи, коди, ETag)
- RFC 9457 — Problem Details · RFC 6902 / 7386 — PATCH
- RFC 8288 — Web Linking · RFC 8594 — Sunset
- github.com/microsoft/api-guidelines
- Fielding, розділ 5 — ics.uci.edu/~fielding/pubs/dissertation

Дякую за увагу

Запитання?

Практика: 21 приклад в одному solution — запускайте, читайте NOTES.md, експериментуйте.